Precise Type Analysis on JavaScript Language Specifications via Generalized Occurrence Typing

Technical Report

ANONYMOUS AUTHOR(S)

This technical report accompanies the paper of the same title. It presents the type analysis of Section 3 of the paper in full (§1): the concrete semantics of $\mathsf{IR}_{\mathsf{ES}}$, the type operations, the lattice operations on symbolic types, the concretization of the abstract domain, and the soundness theorem with its proof structure. It also defines the provenance tracking (§2) and the base type analysis that our analysis extends (§3). The concrete semantics, the analysis, and the proof are mechanized in Lean 4.

1 Type Analysis with Generalized Occurrence Typing

This section defines our type analysis as an abstract interpretation [1, 2] of $\mathsf{IR}_{\mathsf{ES}}$, the untyped intermediate representation for ECMA-262 [4]. It extends the base type analysis of JSTAR [3], which refines types only with the syntax of conditions, in four ways. 1) An abstract value is a symbolic type with a type guard instead of a type. 2) A typing result carries a type proposition that holds at the current point. 3) A branch refines types with the type guard of its condition. 4) The analysis infers the write effect of each function, and a call instantiates the type guard that the callee returns and weakens the type propositions of the caller with the write effect of the callee. Figures 5 and 7 define new operations, and the other figures highlight the modified parts in green. This report gives the full concrete semantics (§1.1), the soundness proof (§1.4), the provenance tracking (§2), and the base analysis that our analysis extends (§3). We define $\mathsf{IR}_{\mathsf{ES}}$ (§1.1), the abstract domains (§1.2), the abstract semantics (§1.3), and the soundness of the analysis (§1.4).

1.1 $\mathsf{IR}_{\mathsf{ES}}$: Untyped Intermediate Representation for ECMA-262

An $\mathsf{IR}_{\mathsf{ES}}$ program is a mechanized specification and its instructions are uniquely labeled with $l \in \mathcal{L}$.

Syntax. Figure 1 shows the syntax of $\mathsf{IR}_{\mathsf{ES}}$. A program P consists of three mappings: 1) $\mathsf{func} : \mathcal{L} \rightharpoonup \mathcal{F}$, 2) $\mathsf{inst} : \mathcal{L} \to \mathcal{I}$, and 3) $\mathsf{next} : \mathcal{L} \rightharpoonup \mathcal{L}$ that map labels to their enclosing functions, instructions, and next labels, respectively, where $\rightharpoonup$ denotes a partial function. An instruction $i \in \mathcal{I}$ is either a sequence, an assignment, a record allocation, a field update, a branch, an assertion, a function call, or a return instruction. An expression $e \in \mathcal{E}$ is either a literal, a variable, a unary/binary operation, a function, or a dynamic type check expression. We write $\overline{X_k}$ for a sequence $X_1, \ldots, X_n$ whose k-th element is X_k ($1 \le k \le n$), or $\overline{X}$ when the index does not matter.

Types. A dynamic type check expression "$e : \tau$" checks if the expression e is an instance of the type τ. A type $\tau \in \mathcal{T}$ is either literal (e.g., `42`, `#t`, `"abc"`), a primitive (**B** for booleans, **N** for numbers, **S** for strings), a function, a record, a union, an intersection, a difference, or a top/bottom type. We define type concretization $\gamma_\tau : \mathcal{T} \to \mathcal{P}(\mathbb{H} \times \mathbb{V})$, subtype relation $<: \subseteq \mathcal{T} \times \mathcal{T}$, field access $\tau.\mathtt{f} \in \mathcal{T}$, and field update $\tau[\mathtt{f} : \tau] \in \mathcal{T}$ as follows.

- **Type Concretization**: $\gamma_\tau : \mathcal{T} \to \mathcal{P}(\mathbb{H} \times \mathbb{V})$

$$
\begin{array}{lll}
\gamma_\tau(p) = \mathbb{H} \times \{p\} & \gamma_\tau(\Lambda) = \mathbb{H} \times \{\Lambda\} & \gamma_\tau(\tau_1 \setminus \tau_2) = \gamma_\tau(\tau_1) \setminus \gamma_\tau(\tau_2) \\
\gamma_\tau(\mathbf{B}) = \mathbb{H} \times \mathbb{B} & \gamma_\tau(\tau_1 \sqcap \tau_2) = \gamma_\tau(\tau_1) \cap \gamma_\tau(\tau_2) & \gamma_\tau(\top) = \mathbb{H} \times \mathbb{V} \\
\gamma_\tau(\mathbf{N}) = \mathbb{H} \times \mathbb{N} & \gamma_\tau(\tau_1 \sqcup \tau_2) = \gamma_\tau(\tau_1) \cup \gamma_\tau(\tau_2) & \gamma_\tau(\bot) = \varnothing
\end{array}
$$

$$
\begin{array}{lll}
\textbf{Programs} & P & = (\text{func}, \text{inst}, \text{next}) \\
\textbf{Instructions} & \mathcal{I} \ni i & ::= x = e \mid x = \{\overline{\mathtt{f} : e}\} \mid e.\mathtt{f} = e \\
 & & \mid \mathtt{if}\ (e)\ l\ \mathtt{else}\ l \mid \mathtt{assert}\ e \\
 & & \mid x = e(\overline{e}) \mid \mathtt{return}\ e \\
\textbf{Expressions} & \mathcal{E} \ni e & ::= p \mid x \mid e.\mathtt{f} \mid \ominus e \mid e \oplus e \mid \Lambda \mid e : \tau \\
\textbf{Types} & \mathcal{T} \ni \tau & ::= p \mid \mathbf{B} \mid \mathbf{N} \mid \mathbf{S} \mid \Lambda \mid \{\overline{\mathtt{f} : \tau}\} \\
 & & \mid \tau \sqcup \tau \mid \tau \sqcap \tau \mid \tau \backslash \tau \mid \top \mid \bot
\end{array}
$$

$$
\begin{array}{lll}
\textbf{Functions} & \mathcal{F} \ni \Lambda & ::= \lambda(\overline{x}).l \\
\textbf{Labels} & \mathcal{L} \ni l & \\
\textbf{Variables} & \mathcal{X} \ni x & \\
\textbf{Fields} & \mathbb{F} \ni \mathtt{f} & \\
\textbf{Primitives} & \mathbb{P} \ni p & ::= b \mid n \mid s \\
\textbf{Booleans} & \mathbb{B} \ni b & ::= \#\mathtt{t} \mid \#\mathtt{f} \\
\textbf{Numbers} & \mathbb{N} \ni n & \\
\textbf{Strings} & \mathbb{S} \ni s &
\end{array}
$$

Fig. 1. Syntax of the *imperative*-style untyped intermediate representation IR_{ES} for ECMA-262.

$$
\begin{array}{llll}
\textbf{States} & \sigma \in \Sigma = \mathbb{K} \times \mathcal{L} \times \mathbb{M} & \textbf{Calling Contexts} & \kappa \in \mathbb{K} = \{\epsilon\} \uplus \Sigma \\
\textbf{Memories} & m \in \mathbb{M} = \mathbb{H} \times \mathbb{E} & \textbf{Heaps} & h \in \mathbb{H} = \mathbb{A} \rightharpoonup \mathbb{R} \\
\textbf{Environments} & \rho \in \mathbb{E} = \mathcal{X} \rightharpoonup \mathbb{V} & \textbf{Addresses} & a \in \mathbb{A} \\
\textbf{Values} & v \in \mathbb{V} = \mathbb{P} \uplus \mathbb{A} \uplus \mathcal{F} & \textbf{Records} & r \in \mathbb{R} = \mathbb{F} \rightharpoonup \mathbb{V}
\end{array}
$$

Fig. 2. States of IR_{ES} programs.

$$
\begin{aligned}
\gamma_\tau(\{\overline{\mathtt{f}_k : \tau_k}\}) = \{(h, a) \mid\ & a \in \mathbb{A} \wedge h \in \mathbb{H} \wedge h(a) = \{\overline{\mathtt{f}'_i : v_i}\} \\
& \wedge\ \forall k. \exists i. (\mathtt{f}_k = \mathtt{f}'_i \wedge (h, v_i) \in \gamma_\tau(\tau_k))\}
\end{aligned}
$$

- **Subtype Relation**: $<: \subseteq \mathcal{T} \times \mathcal{T}$

$$\tau_1 <: \tau_2 \iff \gamma_\tau(\tau_1) \subseteq \gamma_\tau(\tau_2)$$

- **Field Access**: $\tau.\mathtt{f} \in \mathcal{T}$

$$
\begin{array}{llll}
p.\mathtt{f} = \top & \{\overline{\mathtt{f}_k : \tau_k}\}.\mathtt{f} = \begin{cases} \tau_k & \text{if } \mathtt{f} = \mathtt{f}_k \\ \top & \text{otherwise} \end{cases} & (\tau_1 \sqcup \tau_2).\mathtt{f} = \tau_1.\mathtt{f} \sqcup \tau_2.\mathtt{f} & \Lambda.\mathtt{f} = \top \\
\mathbf{B}.\mathtt{f} = \top & & (\tau_1 \sqcap \tau_2).\mathtt{f} = \tau_1.\mathtt{f} \sqcap \tau_2.\mathtt{f} & \top.\mathtt{f} = \top \\
\mathbf{N}.\mathtt{f} = \top & & (\tau_1 \backslash \tau_2).\mathtt{f} = \tau_1.\mathtt{f} & \bot.\mathtt{f} = \bot
\end{array}
$$

- **Field Update**: $\tau[\mathtt{f} : \tau] \in \mathcal{T}$

$$
\begin{array}{lll}
p[\mathtt{f} : \tau] = \bot & (\tau_1 \sqcup \tau_2)[\mathtt{f} : \tau_f] = \tau_1[\mathtt{f} : \tau_f] \sqcup \tau_2[\mathtt{f} : \tau_f] & \Lambda[\mathtt{f} : \tau] = \bot \\
\mathbf{B}[\mathtt{f} : \tau] = \bot & (\tau_1 \sqcap \tau_2)[\mathtt{f} : \tau_f] = \tau_1[\mathtt{f} : \tau_f] \sqcap \tau_2[\mathtt{f} : \tau_f] & \top[\mathtt{f} : \tau] = \{\mathtt{f} : \tau\} \\
\mathbf{N}[\mathtt{f} : \tau] = \bot & (\tau_1 \backslash \tau_2)[\mathtt{f} : \tau_f] = \tau_1[\mathtt{f} : \tau_f] \backslash \tau_2[\mathtt{f} : \tau_f] & \bot[\mathtt{f} : \tau] = \bot
\end{array}
$$

States. A program state $\sigma \in \Sigma$ is a tuple of a calling context, a label, and a memory. A calling context $\kappa \in \mathbb{K}$ is an empty context ϵ or the state at the call site. A memory $m \in \mathbb{M} = \mathbb{H} \times \mathbb{E}$ is a pair of a heap and an environment. A heap $h \in \mathbb{H}$ is a finite map from addresses $a \in \mathbb{A}$ to records, a record $r \in \mathbb{R}$ is a finite map from fields to values, and an environment $\rho \in \mathbb{E}$ is a finite map from variables to values. A value $v \in \mathbb{V}$ is a primitive, an address, or a function. Variables and fields are *mutable*, and function values do not capture the current environment.

Semantics. The concrete semantics is a transition relation $\rightsquigarrow \subseteq \Sigma \times \Sigma$ that executes the instruction at the label of a state and moves to the next label, the callee, or the return site. The *collecting semantics* $[\![P]\!]$ is the set of states reachable from a set of initial states Σ_ι by this relation:

$$[\![P]\!] = \{\sigma \in \Sigma \mid \exists \sigma_\iota \in \Sigma_\iota.\ \sigma_\iota \rightsquigarrow^* \sigma\}$$

$$
\begin{array}{lll}
[\![x = e]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m_1) & \text{where } m_1 = m.\mathsf{update}_x(x, [\![e]\!]_e(m)) \\
[\![x = \{\overline{\mathtt{f}_k : e_k}\}]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m_2) & \text{where } \begin{cases} (a, m_1) = m.\mathsf{alloc}(\{\overline{\mathtt{f}_k : [\![e_k]\!]_e(m)}\}) \\ m_2 = m_1.\mathsf{update}_x(x, a) \end{cases} \\
[\![e_1.\mathtt{f} = e_2]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m_1) & \text{where } m_1 = m.\mathsf{update}_{\mathtt{f}}([\![e_1]\!]_e(m), \mathtt{f}, [\![e_2]\!]_e(m)) \\
[\![\mathtt{if}\ (e)\ l_1\ \mathtt{else}\ l_2]\!]_i(\sigma) & = (\kappa, l', m) & \text{where } l' = \begin{cases} l_1 \text{ if } [\![e]\!]_e(m) = \mathtt{\#t} \\ l_2 \text{ if } [\![e]\!]_e(m) = \mathtt{\#f} \end{cases} \\
[\![\mathtt{assert}(e)]\!]_i(\sigma) & = (\kappa, \mathsf{next}(l), m) & \text{if } \quad [\![e]\!]_e(m) = \mathtt{\#t} \\
[\![\mathtt{call}\ x = e_0(\overline{e_k})]\!]_i(\sigma) & = (\kappa_0, l_0, m_1) & \text{where } \begin{cases} \kappa_0 = \sigma \\ [\![e_0]\!]_e(m) = \lambda(\overline{x_k}).l_0 \\ m_1 = (h, \{\overline{x_k \mapsto [\![e_k]\!]_e(m)}\}) \end{cases} \\
[\![\mathtt{return}\ e]\!]_i(\sigma) & = (\kappa_1, \mathsf{next}(l_1), m_3,) & \text{where } \begin{cases} \kappa = (\kappa_1, l_1, (_, \rho_1)) \\ \mathsf{inst}(l_1) = \mathtt{call}\ x = _; \\ m_2 = (h, \rho_1) \\ m_3 = m_2.\mathsf{update}_x(x, [\![e]\!]_e(m)) \end{cases}
\end{array}
$$

$$
\begin{array}{lll}
[\![p]\!]_e(m) = p & [\![e.\mathtt{f}]\!]_e(m) = h([\![e]\!]_e(m))(\mathtt{f}) & \\
[\![x]\!]_e(m) = \rho(x) & [\![\ominus\ e]\!]_e(m) = \ominus\ [\![e]\!]_e(m) & [\![e : \tau]\!]_e(m) = \begin{cases} \mathtt{\#t} \text{ if } (h, [\![e]\!]_e(m)) \in \gamma_\tau(\tau) \\ \mathtt{\#f} \text{ otherwise} \end{cases} \\
[\![\Lambda]\!]_e(m) = \Lambda & [\![e_1 \oplus e_2]\!]_e(m) = [\![e_1]\!]_e(m) \oplus [\![e_2]\!]_e(m) &
\end{array}
$$

Fig. 3. Semantics of instructions and expressions of $\mathsf{IR}_{\mathsf{ES}}$ where $\sigma = (\kappa, l, m)$ and $m = (h, \rho)$.

Figure 3 defines the transition rules of instructions $[\![-]\!]_i : \Sigma \to \Sigma$ and expressions $[\![-]\!]_e : \mathbb{M} \to \mathbb{V}$ with the following auxiliary operations:

$$
\begin{array}{llll}
\sigma.\mathsf{caller} = \kappa & \kappa.\mathsf{state} = \sigma & m.\mathsf{heap} & = h \\
\sigma.\mathsf{label} = l & \kappa.\mathsf{caller} = \sigma.\mathsf{caller} & m.\mathsf{env} & = \rho \\
\sigma.\mathsf{mem} = m & \kappa.\mathsf{label} = \sigma.\mathsf{label} & m.\mathsf{update}_x(x, v) & = (h, \rho[x \mapsto v]) \\
\sigma.\mathsf{heap} = m.\mathsf{heap} & \kappa.\mathsf{mem} = \sigma.\mathsf{mem} & m.\mathsf{update}_{\mathtt{f}}(a, \mathtt{f}, v) & = (h[a \mapsto h(a)[\mathtt{f} \mapsto v]], \rho) \\
\sigma.\mathsf{env} = m.\mathsf{env} & \kappa.\mathsf{heap} = \sigma.\mathsf{heap} & m.\mathsf{alloc}(r) & = (a, (h[a \mapsto r], \rho)) \\
\text{where } \sigma = (\kappa, l, m) & \kappa.\mathsf{env} = \sigma.\mathsf{env} & & \text{where } a \text{ is a fresh address in } h \\
 & \kappa.\mathsf{var} = x & \text{where } m = (h, \rho) & \\
 & \text{where } \kappa = (\sigma, x) & &
\end{array}
$$

1.2 Abstract Domains

Figure 4 shows the abstract domains. An abstract state $\widehat{\sigma} \in \widehat{\Sigma}$ is a pair of an abstract calling context and a flow-sensitive typing result. An abstract calling context $\widehat{\kappa} \in \widehat{\mathbb{K}}$ maps each function to a set of call-site labels, and a flow-sensitive typing result $\xi \in \Xi$ maps labels to typing results. A typing result $\Gamma = (\widehat{\rho}, \psi, \delta) \in \mathbb{\Gamma}$ consists of an abstract environment $\widehat{\rho}$ from variables to abstract values, a type proposition ψ that always holds at the current point, and a write effect δ from fields to types that records which fields the current function may have written and with which types. In the base analysis, an abstract value is a type, and a typing result has neither a type proposition nor an effect. In our analysis, an abstract value $\widehat{v} \in \widehat{\mathbb{V}}$ is a symbolic type $\tau^\omega \in \mathcal{T}^\omega$ with a type guard $G \in \mathbb{G}$, written $\tau^\omega[G]$. We introduce type propositions, symbolic types, and type guards in turn.

1.2.1 Type Propositions. A *type proposition* $\psi \in \Psi$ is a proposition on the types of variables and symbols. A *symbol* (or a *type variable*) $\omega \in \Omega$ stands for the value of an argument passed by a caller; ω^0, ω^1, ...stand for the first, second, and so on arguments. The top ($\top_\psi$) and bottom ($\bot_\psi$) type propositions always hold and never hold, respectively. For example, a type proposition

Abstract States	$\widehat{\sigma}$	$\in \widehat{\Sigma}$	$=$	$\widehat{\mathbb{K}} \times \Xi$
Abstract Calling Contexts	$\widehat{\kappa}$	$\in \widehat{\mathbb{K}}$	$=$	$\mathcal{F} \to \mathcal{P}(\mathcal{L})$
Flow-Sensitive Typing Results	ξ	$\in \Xi$	$=$	$\mathcal{L} \to \mathbb{\Gamma}$
Typing Results	Γ	$\in \mathbb{\Gamma}$	$=$	$\widehat{\mathbb{E}} \times \Psi \times \Delta$
Abstract Environments	$\widehat{\rho}$	$\in \widehat{\mathbb{E}}$	$=$	$\mathcal{X} \to \widehat{\mathbb{V}}$
Abstract Values	$\widehat{v}$	$\in \widehat{\mathbb{V}}$	$=$	$\mathcal{T}^{\omega} \times \mathbb{G}$
Type Propositions	ψ	$\in \Psi$	$::=$	$\top_{\psi} \mid \bot_{\psi} \mid x : \tau \mid \omega <: \tau \mid \psi \wedge \psi \mid \ldots$
Symbols (Type Variables)	ω	$\in \Omega$	$=$	$\{\omega^{0}, \omega^{1}, \ldots\}$
Symbolic Types	τ^{ω}	$\in \mathcal{T}^{\omega}$	$::=$	$\tau \mid \omega \mid \tau^{\omega}.\mathtt{f} \mid \{\overline{\mathtt{f} : \tau^{\omega}}\}$
Type Guards	G	$\in \mathbb{G}$	$=$	$\mathcal{T}^{T} \to \Psi$
Target Types	τ^{T}	$\in \mathcal{T}^{T}$	$\subset$	$\mathcal{T}$
Effects	δ	$\in \Delta$	$=$	$\mathbb{F} \to \mathcal{T}$

Fig. 4. Abstract domains of the type analysis.

$$\frac{}{\lceil \tau \rceil_{\psi} = \tau} \qquad \frac{\psi \Rightarrow (\omega <: \tau)}{\lceil \omega \rceil_{\psi} = \tau} \qquad \frac{\lceil \tau^{\omega} \rceil_{\psi} = \tau}{\lceil \tau^{\omega}.\mathtt{f} \rceil_{\psi} = \tau.\mathtt{f}} \qquad \frac{\overline{\lceil \tau_k^{\omega} \rceil_{\psi} = \tau_k}}{\lceil \{\overline{\mathtt{f}_k : \tau_k^{\omega}}\} \rceil_{\psi} = \{\overline{\mathtt{f}_k : \tau_k}\}}$$

Fig. 5. Upper type bound $\lceil \tau^{\omega} \rceil_{\psi}$ of a symbolic type τ^{ω} with a type proposition ψ.

$$\tau_1^{\omega}|_{\psi_1} \sqsubseteq \tau_2^{\omega}|_{\psi_2} \Leftrightarrow \begin{cases} \lceil \tau_1^{\omega} \rceil_{\psi_1} <: \tau_2 & \text{if } \tau_2^{\omega} = \tau_2 \\ \tau_1^{\omega} = \tau_2^{\omega} & \text{otherwise} \end{cases} \qquad \tau_1^{\omega}|_{\psi_1} \sqcup \tau_2^{\omega}|_{\psi_2} = \begin{cases} \tau_1^{\omega} & \text{if } \tau_1^{\omega} = \tau_2^{\omega} \\ \lceil \tau_1^{\omega} \rceil_{\psi_1} \sqcup \lceil \tau_2^{\omega} \rceil_{\psi_2} & \text{otherwise} \end{cases}$$

$$\tau_1^{\omega}|_{\psi_1} \sqcap \tau_2^{\omega}|_{\psi_2} = \begin{cases} \tau_1^{\omega} & \text{if } \tau_1^{\omega} \sqsubseteq \tau_2^{\omega} \\ \tau_2^{\omega} & \text{if } \tau_2^{\omega} \sqsubseteq \tau_1^{\omega} \\ \lceil \tau_1^{\omega} \rceil_{\psi_1} \sqcap \lceil \tau_2^{\omega} \rceil_{\psi_2} & \text{otherwise} \end{cases} \qquad \tau_1^{\omega}|_{\psi_1} \downarrow \tau_2 = \begin{cases} \tau_1^{\omega} & \text{if } \tau_1^{\omega} \sqsubseteq \tau_2^{\omega} \\ \lceil \tau_1^{\omega} \rceil_{\psi_1} \sqcap \tau_2^{\omega} & \text{otherwise} \end{cases}$$

Fig. 6. Partial order ($\sqsubseteq$), join ($\sqcup$), meet ($\sqcap$), and refinement ($\downarrow$) for symbolic types τ^{ω} with type propositions.

$\psi = (x : \mathbf{N}) \wedge (\omega^{0} <: \mathbf{S})$ denotes that the variable x has the number type **N** and the first argument has a type below the string type **S**. We use only conjunctions of type propositions on variables or symbols for simplicity and efficiency of analysis in this paper, but other operations (e.g., disjunctions) or other kinds of type propositions can be freely added. Type propositions are inferred from dynamic type check expressions $e : \tau$ and from user-defined rules for specific expressions (§4.4 of the paper), and the analysis refines the types of variables and symbols with them. Since refining a variable must also refine its aliases, we represent aliases with symbolic types.

1.2.2 Symbolic Types. A *symbolic type* $\tau^{\omega} \in \mathcal{T}^{\omega}$ is either a concrete type τ, a symbol ω, a symbolic field access $\tau^{\omega}.\mathtt{f}$, or a symbolic record type $\{\overline{\mathtt{f} : \tau^{\omega}}\}$. Symbolic types that share the same symbols can be refined together using the type propositions on the symbols. For example, two variables in the following abstract environment $\widehat{\rho} \in \widehat{\mathbb{E}}$ are refined together when the type proposition ψ_1 is

$$
\begin{array}{ll}
\textbf{Lookup} & G(\tau) = \bigsqcap \{\psi \mid (\tau^T \Rightarrow \psi) \in G \wedge (\tau <: \tau^T)\} \\
\textbf{Update} & G[\tau \Rightarrow \psi] = \lambda \tau^T.\ \begin{cases} G(\tau^T) \sqcap \psi & \text{if } \tau <: \tau^T \\ G(\tau^T) & \text{otherwise} \end{cases} \\
\textbf{Refinement} & G \downarrow \tau = \lambda \tau^T.\ \begin{cases} \bot_\psi & \text{if } \tau \sqcap \tau^T = \bot_\tau \\ G(\tau^T) & \text{otherwise} \end{cases} \\
\textbf{Field Lookup} & G.\mathtt{f} = \lambda \tau^T.\ G(\{\mathtt{f} : \tau^T\}) \\
\textbf{Record} & \{\overline{\mathtt{f}_j : G_j}\} = \lambda \tau^T.\ \bigwedge_j G_j(\tau^T.\mathtt{f}_j)
\end{array}
$$

Fig. 7. Operations on type guards $G \in \mathbb{G}$.

refined into ψ_2 because they share the same symbol ω:

$$
\widehat{\rho} = [\ x \mapsto \omega,\ y \mapsto \{\mathtt{f} : \omega\}\ ] \quad \text{means} \quad \begin{cases} [\ x \mapsto \mathbf{N} \sqcup \mathbf{S},\quad y \mapsto \{\mathtt{f} : \mathbf{N} \sqcup \mathbf{S}\}\ ] & \text{if } \psi_1 = \omega <: \mathbf{N} \sqcup \mathbf{S} \\ [\ x \mapsto \mathbf{N},\quad y \mapsto \{\mathtt{f} : \mathbf{N}\}\ ] & \text{if } \psi_2 = \omega <: \mathbf{N} \end{cases}
$$

In addition, parametric polymorphism is supported by instantiating symbols inside symbolic types with concrete actual types at the call-sites. Figure 5 defines the upper type bound $\lceil \tau^\omega \rceil_\psi$ of a symbolic type τ^ω with the current type proposition ψ. The lattice operations ($\sqsubseteq$, $\sqcup$, and $\sqcap$) compare and combine upper type bounds unless the symbolic types are equal, and the refinement $\tau^\omega|_\psi \downarrow \tau$ narrows the upper type bound of τ^ω to τ unless τ^ω is already below τ (Figure 6). Note that $\lceil \tau^\omega \rceil_\psi$ is an upper bound, not the actual type of τ^ω, because a symbol can be instantiated with any type that satisfies ψ.

1.2.3 Type Guards. A *type guard* $G \in \mathbb{G}$ is a mapping from user-chosen target types $\mathcal{T}^T$ to type propositions Ψ. A target type τ^T describes an outcome of an expression. A mapping $\tau^T \Rightarrow \psi$ in the type guard of an expression states that the type proposition ψ holds whenever the expression evaluates to a value of type τ^T. Type guards allow additional type propositions to be applied sequentially, thereby enabling chained application of multiple propositions. For example, consider an abstract environment $\widehat{\rho}$ and its updated one $\widehat{\rho}'$ with a type proposition $\psi_1 = (x : \mathbf{N})$:

$$
\widehat{\rho} = [\ x \mapsto (\mathbf{B} \sqcup \mathbf{N})\ [\mathbf{N} \Rightarrow \overline{(y : \mathbf{S})}^{\,\psi_2}],\quad y \mapsto \mathbf{B} \sqcup \mathbf{S}\ ] \quad \xrightarrow{\text{applying } \overline{(x : \mathbf{N})}^{\,\psi_1}} \quad [\ x \mapsto \mathbf{N}\ [\ldots],\quad y \mapsto \mathbf{S}\ ]
$$

Then, the type of x is refined from $\mathbf{B} \sqcup \mathbf{N}$ to $\mathbf{N}$, which is a subtype of the target type $\mathbf{N}$ in the type guard for x. Therefore, another type proposition $\psi_2 = (y : \mathbf{S})$ in the guard also holds and refines the type of y from $\mathbf{B} \sqcup \mathbf{S}$ to $\mathbf{S}$. Finally, two applied type propositions $\psi_1 = (x : \mathbf{N})$ and $\psi_2 = (y : \mathbf{S})$ are merged into the type proposition ψ' of the updated typing result $\Gamma' = (\widehat{\rho}', \psi', \delta')$.

Operations. The lattice operations ($\sqsubseteq$, $\sqcup$, and $\sqcap$) on type guards are defined in a point-wise manner, and other operations on type guards are defined in Figure 7. The lookup $G(\tau)$ meets the type propositions of all target types that contain τ, and the update $G[\tau \Rightarrow \psi]$ adds ψ to all of them. The refinement $G \downarrow \tau$ sets the type propositions of target types disjoint from τ to $\bot_\psi$.

These operations handle the three problems in §2.3.1 of the paper. Overlapping target types stay consistent because a lookup and an update visit all of them. A lookup ignores a target type narrower than τ because a value of type τ may fall outside it, and it yields $\top_\psi$ when no target type contains τ. A refinement makes the entries of impossible target types $\bot_\psi$, so joining the type guards of several returns keeps what each target type learns from the returns that can produce it. For

example, steps 1 and 4 of **IsArray** in §2.3.1 of the paper return `false`, whose type is disjoint from `Normal[True]`, so they do not weaken the type proposition that step 2 gives to `Normal[True]`.

The field lookup $G.\mathtt{f}$ and record $\{\overline{\mathtt{f} : G}\}$ operations produce a new type guard for the result of a field lookup and a record allocation, respectively. Both are defined in a *query form*: for each target type τ^T of the result, they ask which proposition the original guard promises. If the field value $h(a)(\mathtt{f})$ belongs to τ^T, the base record a belongs to the open record type $\{\mathtt{f} : \tau^T\}$, so $G.\mathtt{f}$ looks up the base guard G with this record type. Dually, if an allocated record belongs to τ^T, each field value belongs to $\tau^T.\mathtt{f}_j$, so the record guard conjoins the lookups of the field guards. Note that the type guard of the field value cannot inherit a proposition for a target type with multiple fields (e.g., $\{\mathtt{f} : \mathbf{N}, \mathtt{g} : \mathbf{S}\}$) because the field value alone does not determine the other fields. Such a proposition remains only in the type guard of the base record. Field updates do not have a dedicated guard operation; they are handled by effect-based weakening (Figure 11).

When a type guard G is bound to a specific symbolic type τ^ω in an abstract value (i.e., $\widehat{v} = \tau^\omega[G]$), the refinement operation is used for removing infeasible propositions (i.e., $G \downarrow \lceil \tau^\omega \rceil_\psi$). In practice, the entries of a type guard that are $\bot_\psi$ by the refinement or equal to the current type proposition ψ need not be stored. If an abstract value $\widehat{v} = \tau^\omega[G]$ has a top type guard $G = \top_G$, we simply write τ^ω instead of $\tau^\omega[\top_G]$. The lattice operations on abstract values and the other abstract domains are point-wise. For example, $\{\overline{\mathtt{f}_k : \widehat{v}_k}\} = \{\overline{\mathtt{f}_k : \tau_k^\omega}\}\ [\{\overline{\mathtt{f}_k : G_k}\}]$ where $\widehat{v}_k = \tau_k^\omega[G_k]$.

Example. Consider the following type guard G with five target types $\mathcal{T}^T = \{\#\mathtt{t}, \#\mathtt{f}, \mathbf{B}, \mathbf{N}, \{\mathtt{f} : \mathbf{N}\}\}$:

$$G = [\ \#\mathtt{t} \Rightarrow \psi_{\#\mathtt{t}},\quad \#\mathtt{f} \Rightarrow \psi_{\#\mathtt{f}},\quad \mathbf{B} \Rightarrow \psi_{\mathbf{B}},\quad \mathbf{N} \Rightarrow \psi_{\mathbf{N}}\quad \{\mathtt{f} : \mathbf{N}\} \Rightarrow \psi_{\mathtt{f}:\mathbf{N}}\ ]$$

Then, the result of each operation on the type guard G is as follows:

$$\begin{array}{lll}
G(\#\mathtt{t}) & = \psi_{\#\mathtt{t}} \sqcap \psi_{\mathbf{B}} & \\
G[\#\mathtt{t} \Rightarrow \psi] & = [\ \#\mathtt{t} \Rightarrow \psi_{\#\mathtt{t}} \sqcap \psi, \#\mathtt{f} \Rightarrow \psi_{\#\mathtt{f}}, \mathbf{B} \Rightarrow \psi_{\mathbf{B}} \sqcap \psi, \mathbf{N} \Rightarrow \psi_{\mathbf{N}}\quad \{\mathtt{f} : \mathbf{N}\} \Rightarrow \psi_{\mathtt{f}:\mathbf{N}}\ ] \\
G \downarrow \mathbf{B} & = [\ \#\mathtt{t} \Rightarrow \psi_{\#\mathtt{t}}, \quad \#\mathtt{f} \Rightarrow \psi_{\#\mathtt{f}}, \mathbf{B} \Rightarrow \psi_{\mathbf{B}}, \quad \mathbf{N} \Rightarrow \bot_\psi\ \ \{\mathtt{f} : \mathbf{N}\} \Rightarrow \bot_\psi\ ] \\
G.\mathtt{f} & = [\ \#\mathtt{t} \Rightarrow \top_\psi, \quad \#\mathtt{f} \Rightarrow \top_\psi, \mathbf{B} \Rightarrow \top_\psi, \quad \mathbf{N} \Rightarrow \psi_{\mathtt{f}:\mathbf{N}}\ \{\mathtt{f} : \mathbf{N}\} \Rightarrow \top_\psi\ ] \\
\{\mathtt{f} : G\} & = [\ \#\mathtt{t} \Rightarrow \top_\psi, \quad \#\mathtt{f} \Rightarrow \top_\psi, \mathbf{B} \Rightarrow \top_\psi, \quad \mathbf{N} \Rightarrow \top_\psi,\ \ \{\mathtt{f} : \mathbf{N}\} \Rightarrow \psi_{\mathbf{N}}\ ]
\end{array}$$

1.3 Abstract Semantics

The abstract semantics $\widehat{[\![P]\!]}$ of a program is the least fixpoint of the abstract transfer function $\widehat{F} : \widehat{\Sigma} \to \widehat{\Sigma}$ from an initial abstract state $\widehat{\sigma}_\iota$ that covers the initial states Σ_ι:

$$\widehat{[\![P]\!]} = \mathbf{lfp}\widehat{F} \qquad \text{where} \qquad \widehat{F}(\widehat{\sigma}) = \widehat{\sigma}_\iota \sqcup \left(\bigsqcup_{l \in \mathcal{L}} \widehat{[\![\mathsf{inst}(l)]\!]}_i(l, \widehat{\sigma}) \right)$$

Figures 8, 9, and 10 define the abstract semantics $\widehat{[\![-]\!]}_e$ of expressions and $\widehat{[\![-]\!]}_i$ of instructions. Type propositions are 1) *inferred* when analyzing expressions or producing abstract values, 2) *propagated* into other program points across functions, 3) used to *refine* types of variables and symbols in branch or assertion instructions, and 4) *weakened* when variables or fields are mutated.

1.3.1 Type Proposition Inference. Figure 8 defines the abstract semantics $\widehat{[\![-]\!]}_e$ of expressions as in the base analysis and its *guarded* version $\widehat{[\![-]\!]}^{\mathrm{g}}_e$, which attaches a type guard to the result and binds the current type proposition ψ to it, and the instruction rules use the latter.

$$\widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma) = \widehat{[\![e]\!]}_e(\Gamma)[\,G\,].\,\widehat{\mathbf{bind}}(\psi) \text{ where } G = \widehat{\mathbf{infer}}_G(\Gamma, e)$$

$$\widehat{[\![p]\!]}_e(\Gamma) = p \qquad \widehat{[\![x]\!]}_e(\Gamma) = \widehat{\rho}(x) \qquad \widehat{[\![\ominus\, e]\!]}_e(\Gamma) = \ominus \widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma) \qquad \widehat{[\![e : \tau]\!]}_e(\Gamma) = \mathbf{B}$$

$$\widehat{[\![\Lambda]\!]}_e(\Gamma) = \Lambda \qquad \widehat{[\![e.\mathtt{f}]\!]}_e(\Gamma) = \widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma).\mathtt{f} \qquad \widehat{[\![e_1 \oplus e_2]\!]}_e(\Gamma) = \widehat{[\![e_1]\!]}^{\mathrm{g}}_e(\Gamma) \oplus \widehat{[\![e_2]\!]}^{\mathrm{g}}_e(\Gamma)$$

Fig. 8. Abstract semantics $\widehat{[\![-]\!]}_e : \mathbb{\Gamma} \to \widehat{\mathbb{V}}$ of expressions and its guarded version $\widehat{[\![-]\!]}^{\mathrm{g}}_e$ where $\Gamma = (\widehat{\rho}, \psi, \delta)$ and the operations on types are lifted to symbolic types.

Inference. The $\widehat{\mathbf{infer}}_G$ operation infers a type guard for a dynamic type check expression $e : \tau$ as follows, and it gives the top type guard $\top_G$ for the other expressions:

$$\widehat{\mathbf{infer}}_G(\Gamma, e : \tau) = \top_G[\mathtt{\#t} \Rightarrow \psi_{\mathtt{\#t}}][\mathtt{\#f} \Rightarrow \psi_{\mathtt{\#f}}] \quad \text{where} \quad \begin{cases} (\tau^\omega, _) = \widehat{[\![e]\!]}_e(\Gamma) \\ \psi_{\mathtt{\#t}} = \widehat{\mathbf{infer}}_\psi(e, \tau^\omega)(\lceil \tau^\omega \rceil_\psi \sqcap \tau) \\ \psi_{\mathtt{\#f}} = \widehat{\mathbf{infer}}_\psi(e, \tau^\omega)(\lceil \tau^\omega \rceil_\psi \setminus \tau) \end{cases}$$

It first evaluates e to its symbolic type τ^ω. Then, it infers two type propositions $\psi_{\mathtt{\#t}}$ and $\psi_{\mathtt{\#f}}$ that hold when e is a subtype of τ and disjoint from τ, respectively, with the $\widehat{\mathbf{infer}}_\psi$ operation, and binds them to the target types `#t` and `#f`:

$$\widehat{\mathbf{infer}}_\psi(e, \tau^\omega)(\tau) = \begin{cases} \psi & \text{if } \tau^\omega \in \mathcal{T} \wedge (e <:? \tau \Downarrow \psi) \\ \psi & \text{if } \tau^\omega \notin \mathcal{T} \wedge (\tau^\omega <:? \tau \Downarrow \psi) \\ \top_\psi & \text{otherwise} \end{cases}$$

where the judgment $e <:? \tau \Downarrow \psi$ (or $\tau^\omega <:? \tau \Downarrow \psi$) reads "if the expression e (or the symbolic type τ^ω) has a type below τ, then the type proposition ψ holds," and it is derived by the following rules:

$$\frac{}{x <:? \tau \Downarrow x : \tau} \qquad \frac{e <:? \{\mathtt{f} : \tau\} \Downarrow \psi}{e.\mathtt{f} <:? \tau \Downarrow \psi} \qquad \frac{}{\omega <:? \tau \Downarrow \omega <: \tau} \qquad \frac{\tau^\omega <:? \{\mathtt{f} : \tau\} \Downarrow \psi}{\tau^\omega.\mathtt{f} <:? \tau \Downarrow \psi} \qquad \frac{\overline{\tau_k^\omega <:? \tau.\mathtt{f}_k \Downarrow \psi_k}}{\{\mathtt{f}_k : \tau_k^\omega\} <:? \tau \Downarrow \bigwedge\{\overline{\psi_k}\}}$$

While this formalization only supports type guards for dynamic type check expressions, we can define additional rules for specific expressions to infer type guards and will define them in §4 of the paper.

Binding. An entry $\tau^T \Rightarrow \psi'$ of a type guard must hold whenever the value has the type τ^T. The current type proposition ψ holds where a value is produced, so it holds for every target type that the value can have. The $\widehat{\mathbf{bind}}$ operation records ψ in the entries of those target types, namely the ones that contain the upper type bound of the value:

$$\widehat{v}.\widehat{\mathbf{bind}}(\psi) = \tau^\omega[G'] \qquad \text{where} \qquad \widehat{v} = \tau^\omega[G] \quad \text{and} \quad G' = G[\lceil \tau^\omega \rceil_\psi \Rightarrow \psi]$$

This is how a type proposition of a branch becomes an entry of a type guard that the value carries to its uses, and $\widehat{\mathbf{bind}}$ is applied to every produced abstract value, such as results of expressions, allocated records, and returned values.

Example. The condition "*argument* is not an Object" in step 1 of **IsArray** is a dynamic type check `argument : (ESValue\Object)` in $\mathsf{IR}_{\mathsf{ES}}$. Since `argument` is the first parameter, its symbolic type is `#0`, and $\widehat{\mathbf{infer}}_G$ gives the type guard of the condition:

```
Boolean { True ⇒ (#0 <: ESValue \ Object), False ⇒ (#0 <: Object) }
```

In the false branch of step 1, the type proposition `(#0 <: Object)` holds, and the return in step 2 produces a value of type `Normal[True]`. Thus, $\widehat{\mathbf{bind}}$ records `(#0 <: Object)` in the

$$
\begin{array}{lll}
\widehat{[\![x = e]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{update}}_x(x, \widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma)) \\
\widehat{[\![x = \{\mathtt{f}_k : e_k\}]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{update}}_x(x, \overline{\{\mathtt{f}_k : \widehat{[\![e_k]\!]}^{\mathrm{g}}_e(\Gamma)\}}.\,\widehat{\mathbf{bind}}(\psi)\,) \\
\widehat{[\![e_1.\mathtt{f} = e_2]\!]}_i(l,\widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{update}}_{\mathtt{f}}(\widehat{v}_1, \mathtt{f}, \widehat{v}_2) \text{ and } \widehat{v}_i = \widehat{[\![e_i]\!]}^{\mathrm{g}}_e(\Gamma) \\
\widehat{[\![\mathtt{call}\ x = e_0(\overline{e_k})]\!]}_i(l,\widehat{\sigma}) & = (\widehat{\kappa}_1, \{\overline{l_j \mapsto \Gamma_j}\}) & \text{where}
\end{array}
$$

$$
\begin{cases}
\widehat{v}_0 & = \widehat{[\![e_0]\!]}_e(\Gamma) \\
(\tau^{\omega}_k, _) & = \widehat{[\![e_k]\!]}_e(\Gamma) \\
\psi' & = \bigwedge\{\overline{\omega^{\mathsf{k}} <: \lceil \tau^{\omega}_k \rceil_{\psi}}\}
\end{cases}
\qquad
\begin{array}{ll}
\{\overline{\Lambda_j}\} & = \{\overline{\lambda(\overline{x_{j,k}}).l_j}\} = \gamma_v(\widehat{v}_0) \cap \mathcal{F} \\
\Gamma_j & = (\{x_{j,k} \mapsto \omega^{\mathsf{k}}\},\ \psi',\ \bot_{\delta}) \\
\widehat{\kappa}_1 & = \{\overline{\Lambda_j \mapsto (l,x)}\}
\end{array}
$$

$$
\widehat{[\![\mathtt{return}\ e]\!]}_i(l,\widehat{\sigma}) = (\varnothing, \{\overline{\mathsf{next}(l_k) \mapsto \Gamma'_k}\}) \quad \text{where}
$$

$$
\begin{cases}
\{\overline{(l_k, x_k)}\} & = \widehat{\kappa} \circ \mathsf{func}(l) \\
\mathsf{inst}(l_k) & = \mathtt{call}\ x_k = _(\overline{e_{k,j}}) \\
\Gamma_k & = \xi(l_k).\,\widehat{\mathbf{apply}}(\delta)
\end{cases}
\qquad
\begin{array}{ll}
\widehat{v} & = \widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma) \\
\widehat{v}'_k & = \widehat{v}.\,\mathbf{instantiate}(\xi(l_k), \overline{e_{k,j}}, \delta) \\
\widehat{v}_k & = \widehat{v}'_k.\,\widehat{\mathbf{bind}}(\Gamma_k.\psi) \\
\Gamma'_k & = \Gamma_k.\widehat{\mathbf{update}}_x(x_k, \widehat{v}_k)
\end{array}
$$

Fig. 9. Abstract semantics $\widehat{[\![-]\!]}_i : (\mathcal{L} \times \widehat{\Sigma}) \to \widehat{\Sigma}$ of assignments, record allocations, field updates, calls, and returns where $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ and $\xi(l) = \Gamma = (\widehat{\rho},\ \psi\ ,\ \delta\ )$.

`Normal[True]` entry of the type guard of the returned value, which gives the return type of **IsArray** in §2.3.1 of the paper:

IsArray(...) : `Normal[Boolean] | Throw { Normal[True] ⇒ (#0 <: Object) }`

1.3.2 Type Proposition Propagation. The inferred type propositions are propagated into other program points, within a function and across functions, stored in the type guards of abstract values. Within a function, the assignment, record allocation, and field update rules in Figure 9 store the guarded abstract values of expressions, so the type propositions of an expression flow with its value into variables and fields. A record allocation is an instruction rather than an expression, so its rule builds the abstract record from the guarded field values, whose type guard is the record type guard in Figure 7, and binds the current type proposition ψ to it as the guarded semantics does for expressions. The update operations $\widehat{\mathbf{update}}_x$ and $\widehat{\mathbf{update}}_{\mathtt{f}}$ also weaken the type propositions that the assignment or the field update invalidates (§1.3.4).

Calls and Returns. When calling a function, the analysis introduces indexed symbols ω^{k} to represent type variables for the actual types of the arguments. The call rule in Figure 9 binds each parameter $x_{j,k}$ to the symbol ω^{k} and starts the callee with the type proposition ψ' that records the upper type bounds of the arguments. Thus, the callee is analyzed once with symbolic arguments, and the type propositions it infers mention the arguments only through symbols. When returning from a function, the $\widehat{\mathbf{apply}}$ operation weakens the typing result $\xi(l_k)$ of the caller at the call site with the callee effect δ (§1.3.4) to obtain Γ_k, which describes what the caller still knows after the call. The returned abstract value is instantiated with the argument expressions evaluated in $\xi(l_k)$ rather than in Γ_k, because each symbol ω^{k} denotes the value of the k-th argument at the call, and the changes since the call are taken into account by the callee effect δ in the instantiation. Finally, the instantiated value is bound with the proposition $\Gamma_k.\psi$ of the caller, not with the proposition of the callee, because the latter may mention local variables of the callee.

$$[\![\widehat{\texttt{if }(e)\ l_1\ \texttt{else}\ l_2}]\!]_i(l,\widehat{\sigma}) = (\varnothing, \{l_t \mapsto \Gamma_t, l_f \mapsto \Gamma_f\}) \text{ where } \begin{cases}\Gamma_t = \Gamma.\widehat{\textbf{refine}}(e, \#\texttt{t})\\ \Gamma_f = \Gamma.\widehat{\textbf{refine}}(e, \#\texttt{f})\end{cases}$$

$$[\![\widehat{\texttt{assert}(e)}]\!]_i(l,\widehat{\sigma}) = (\varnothing, \{\text{next}(l) \mapsto \Gamma_1\}) \quad \text{where } \Gamma_1 = \Gamma.\widehat{\textbf{refine}}(e, \#\texttt{t})$$

Fig. 10. Abstract semantics of branches and assertions where $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ and $\xi(l) = \Gamma = (\widehat{\rho}, \psi, \delta)$.

Instantiation. The **instantiate** operation replaces the symbols in the returned abstract value with the argument expressions and their abstract values:

$$\tau^\omega[G].\textbf{instantiate}(\Gamma, \overline{e_k}, \delta) = \tau_0^\omega[G_0 \sqcap G'] \quad \text{where} \quad \begin{cases}\widehat{v_k} = \tau_k^\omega[G_k] = \textbf{weaken}_\delta(\widehat{[\![e_k]\!]}^{\text{g}}_e(\Gamma))\\ \tau_0^\omega = \tau^\omega[\overline{\omega^{\text{k}} \mapsto \tau_k^\omega}]\\ G_0 = \tau^\omega[\overline{\omega^{\text{k}} \mapsto G_k}]\\ G' = \lambda\tau^T.G(\tau^T).\textbf{instantiate}_\delta([\overline{(e_k, \tau_k^\omega)}])\end{cases}$$

The operation $\psi.\textbf{instantiate}_\delta([\overline{(e_k, \tau_k^\omega)}])$ translates each proposition $\omega^{\text{k}} <: \tau$ on a symbol into the proposition $\textbf{weaken}_\delta(\widehat{\textbf{infer}}_\psi(e_k, \tau_k^\omega)(\tau))$ on the argument expression, distributes over conjunctions, keeps $\bot_\psi$, and maps the other propositions to $\top_\psi$. In general, both type guards 1) G_0 existing in the argument abstract values and 2) G' inferred in the callee are propagated and merged into the type guard of the instantiated returned abstract value. Thus, the result keeps what the arguments already carry and adds what the callee has learned about them.

Effects in Instantiation. A proposition $\omega^{\text{k}} <: \tau$ inferred in the callee is a fact about the *value* of the k-th argument at the call, which never changes. However, its translation $\widehat{\textbf{infer}}_\psi(e_k, \tau_k^\omega)(\tau)$ is a proposition about the argument *expression* e_k in the caller, which is evaluated in the heap at the return. Thus, we weaken the translated proposition with the callee effect δ using $\textbf{weaken}_\delta$ (§1.3.4). For the same reason, the argument values $\widehat{v_k}$ are weakened with $\textbf{weaken}_\delta$, so a field lookup $\tau^\omega.\texttt{f}$ whose field the callee writes is replaced with its upper type bound.

Example. Consider the following callee g and its call:

$$\texttt{g(a, o)} = \{\ \texttt{b} = (\texttt{a} : \mathbf{N});\ \texttt{o.f} = \texttt{"abc"};\ \texttt{return b}\ \} \qquad \texttt{r} = \texttt{g}(x.\texttt{f}, x)$$

at a call site l_k where $\widehat{[\![x.\texttt{f}]\!]}_e(\xi(l_k)) = \mathbf{N}$ before the callee overwrites $x.\texttt{f}$. The returned abstract value is $\widehat{v}_{\text{ret}} = \mathbf{B}[\#\texttt{t} \Rightarrow \omega^{\texttt{0}} <: \mathbf{N}]$ with the callee effect $\delta = [\texttt{f} \mapsto \mathbf{S}]$. The instantiation translates $\omega^{\texttt{0}} <: \mathbf{N}$ into $x : \{\texttt{f} : \mathbf{N}\}$ using $\widehat{\textbf{infer}}_\psi$ and weakens it with δ:

$$\widehat{v}_{\text{ret}}.\textbf{instantiate}(\xi(l_k), [x.\texttt{f}, x], \delta) = \mathbf{B}[\#\texttt{t} \Rightarrow x : \{\texttt{f} : \mathbf{N} \sqcup \mathbf{S}\}]$$

Without the weakening, the guard would wrongly claim that $x.\texttt{f}$ is a number when r is #t. In the same way, the call **IsArray**(element) in **FlattenIntoArray** substitutes the symbolic type of element for #0 in the type guard of **IsArray**, which gives the instantiated type guard in §2.3.1 of the paper, and the empty effect of **IsArray** weakens nothing.

1.3.3 Type Refinement. The base analysis refines types only with the syntax of a condition, such as "C is an Object" in §1 of the paper. Our type refinement in the branch and assertion rules of Figure 10 instead uses the type guard of the condition, so a condition on a variable such as *shouldFlatten* refines other variables such as *element* (§2.3.1 of the paper). It looks up the type guard of the

$$\Gamma.\widehat{\textbf{update}}_x(x,\widehat{v}) = (\widehat{\rho}[x\mapsto\widehat{v}],\psi,\delta).\,\textbf{weaken}_x(x) \qquad \Gamma.\widehat{\textbf{update}}_{\mathsf{f}}(\widehat{v}_1,\mathsf{f},\widehat{v}_2) = \Gamma.\,\widehat{\textbf{apply}}([\mathsf{f}\mapsto\tau_2])$$

Fig. 11. Modified variable and field update operations for **weakening type propositions** where $\Gamma = (\widehat{\rho},\psi,\delta)$, $\widehat{v}_2 = \tau_2^{\omega}[_]$, and $\tau_2 = \textbf{weaken}_{[\mathsf{f}\mapsto\top]}(\lceil\tau_2^{\omega}\rceil_{\psi})$.

condition at the outcome b of the branch and applies the type proposition found there:

$$\Gamma.\widehat{\textbf{refine}}(e,b) = \Gamma.\widehat{\textbf{refine}}(\psi) \qquad \text{where} \qquad _[G] = \widehat{[\![e]\!]}^{\mathrm{g}}_{e}(\Gamma) \quad \text{and} \quad \psi = G[b]$$

Applying a type proposition ψ' refines the typing result as follows:

$$\Gamma.\widehat{\textbf{refine}}(\psi') = \begin{cases} (\widehat{\rho}[x\mapsto\widehat{v}],\psi\wedge\psi',\delta).\widehat{\textbf{refine}}(G[\tau]) & \text{if } (\psi' = x:\tau)\wedge\widehat{v} = (\widehat{\rho}(x)|_{\psi}\downarrow\tau) \\ \Gamma.\widehat{\textbf{refine}}(\psi_0)\sqcap\Gamma.\widehat{\textbf{refine}}(\psi_1) & \text{if } \psi' = (\psi_0\wedge\psi_1) \\ (\widehat{\rho},\psi\wedge\psi',\delta) & \text{otherwise} \end{cases}$$

The first case narrows the abstract value of x with the refinement operation ($\downarrow$) for symbolic types and type guards (Figure 7). It also invokes $\widehat{\textbf{refine}}$ again with the type guard entry $G[\tau]$ of the refined value, which gives the chained application in §1.2.3. For example, the condition "`shouldFlatten` is true" in step 4.c.v of Figure 2 of the paper looks up the type guard of `shouldFlatten` at `True` and narrows `element` to `Object` (§2.3.1 of the paper).

1.3.4 Weakening Type Propositions. The update operations in Figure 11 weaken the type propositions that a mutation may invalidate, and they address the three points in §2.3.2 of the paper.

Variables. After a variable x is assigned a new value, the $\textbf{weaken}_x(x)$ operation replaces every type proposition $x:\tau$ about x with $\top_{\psi}$ in the typing result Γ and its type guards:

$$\Gamma.\textbf{weaken}_x(x) = \Gamma[\top_{\psi}/(x:\tau)]$$

For example, the assignment in step 4.c.iv.1 of Figure 2 of the paper drops the propositions about *shouldFlatten*.

Fields. As in the return rule (§1.3.2), a field update weakens the typing result with an effect δ' using $\Gamma.\widehat{\textbf{apply}}(\delta')$, which weakens every type, proposition, and type guard in Γ with $\textbf{weaken}_{\delta'}$ and accumulates the effect:

$$\Gamma.\widehat{\textbf{apply}}(\delta') = (\{\overline{x\mapsto\textbf{weaken}_{\delta'}(\widehat{v})}\},\textbf{weaken}_{\delta'}(\psi),\delta\fatsemi\delta') \quad \text{where} \quad \widehat{\rho} = \{\overline{x\mapsto\widehat{v}}\}$$

The accumulated effect $(\delta\fatsemi\delta')(\mathsf{f}) = \textbf{weaken}_{\delta'}(\delta(\mathsf{f}))\sqcup\delta'(\mathsf{f})$ weakens the types already recorded in δ by δ' and adds δ', where $\delta(\mathsf{f}) = \bot$ if $\mathsf{f}\notin\mathrm{dom}(\delta)$. Thus, the effect of a function covers its own writes and those of its callees (the second point in §2.3.2 of the paper). The $\textbf{weaken}_{\delta}$ operation extends the type of each field $\mathsf{f}\in\mathrm{dom}(\delta)$ with $\delta(\mathsf{f})$ in every record type, including nested ones. We call a type *heap-dependent* if it contains record types, because its membership can change after a mutation, and *heap-independent* otherwise. With the current type proposition ψ, $\textbf{weaken}_{\delta}$ is defined as follows:

$$\begin{array}{lll} \textbf{weaken}_{\delta}(\{\overline{\mathsf{f}_k:\tau_k^{\omega}}\}) = \{\overline{\mathsf{f}_k:\tau'^{\omega}_k}\} & & \\ \quad \text{where } \tau'^{\omega}_k & = \textbf{weaken}_{\delta}(\lceil\tau_k^{\omega}\rceil_{\psi})\sqcup\delta(\mathsf{f}_k) & \text{if } \mathsf{f}_k\in\mathrm{dom}(\delta), \text{ else } \textbf{weaken}_{\delta}(\tau_k^{\omega}) \\ \textbf{weaken}_{\delta}(\tau^{\omega}.\mathsf{f}) & = \textbf{weaken}_{\delta}(\lceil\tau^{\omega}\rceil_{\psi}.\mathsf{f}) & \text{if } \mathsf{f}\in\mathrm{dom}(\delta), \text{ else } \textbf{weaken}_{\delta}(\tau^{\omega}).\mathsf{f} \\ \textbf{weaken}_{\delta}(\tau_1\setminus\tau_2) & = \textbf{weaken}_{\delta}(\tau_1)\setminus\tau_2 & \text{if } \tau_2 \text{ is heap-independent, else } \textbf{weaken}_{\delta}(\tau_1) \\ \textbf{weaken}_{\delta}(G)(\tau^T) & = \textbf{weaken}_{\delta}(G)(\tau^T) & \text{if } \tau^T \text{ is heap-independent, else } \top_{\psi} \end{array}$$

The first case also covers record types, since a type is a symbolic type. For the other forms, $\mathbf{weaken}_\delta$ keeps symbols and applies to each component, as in $\mathbf{weaken}_\delta(\omega <: \tau) = \omega <: \mathbf{weaken}_\delta(\tau)$. The last two cases drop heap-dependent types in negative positions and the type guard entries of heap-dependent target types, which does not affect the target types used in practice (e.g., `#t`, `#f`, and completion types). The second case replaces a field lookup $\tau^\omega.\mathtt{f}$ with its upper type bound because the field may change, while a symbol ω is stable. Since this formalization uses uniform abstract addresses, a write to a field weakens that field in every record type (the first point in §2.3.2 of the paper), and the implementation partitions addresses by record types (§4.1 of the paper).

Example. In **Map.prototype.get** (§2.2.2 of the paper), the type proposition $M : \{[[\mathtt{MapData}]] : \tau\}$ holds after step 2, where τ is the list type of the field. The callee **CanonicalizeKeyedCollectionKey** in step 3 writes no fields, so its effect is empty. Thus, $\widehat{\textbf{apply}}$ at the return keeps the type proposition, and step 4 reads $M.[[\mathtt{MapData}]]$ as τ without a false alarm. If the callee wrote $[[\mathtt{MapData}]]$ with a value of type τ', its effect $[[[\mathtt{MapData}]] \mapsto \tau']$ would weaken the type proposition into $M : \{[[\mathtt{MapData}]] : \tau \sqcup \tau'\}$, as well as this field in every other record type.

1.4 Soundness

This section defines the concretization of the abstract domain in §1.2 and states the soundness theorem of the type analysis with its proof structure. All definitions and the proof are mechanized in Lean 4 without any admitted lemma in the directory `proof/` of the artifact, whose `README.md` maps each definition and lemma to the Lean code.

Scope of the Mechanization. The mechanization covers the core of $\mathsf{IR}_{\mathsf{ES}}$: expressions (primitives, variables, field lookups, unary and binary operations, functions, and dynamic type checks) and instructions (assignments, record allocations, field updates, branches, assertions, calls, and returns). Unary and binary operators are uninterpreted, and the analysis approximates their results with $\top$. The extended propositions in §4.4 of the paper (e.g., **SameType**), the provenance tracking in §2, and the partitioned abstract addresses in §4 of the paper are not mechanized. For branches, the false branch uses the target type $\top \setminus \mathtt{\#t}$ (non-true) instead of `#f`, following the truthiness of the concrete semantics.

1.4.1 Concrete States. A concrete state $\sigma = (\kappa, l, m)$ consists of a calling context κ, a label l, and a memory $m = (h, \rho)$. A calling context is either empty (ϵ) or the state of the caller $\sigma_c = (\kappa_c, l_c, m_c)$ at its call instruction at l_c. The *entry heap* of a frame is the heap h_c of m_c when the current function was called, or the initial heap h_{init} for the main frame. We write $\sigma \rightsquigarrow \sigma'$ for a transition of the concrete semantics in §1.1.

1.4.2 Concretization. The extended abstract domain refers to symbols and variables, so its concretization is defined relative to a heap h, an environment ρ, and a symbol valuation.

Definition 1.1 (Symbol Valuation). A symbol valuation $\eta : \Omega \to \mathbb{V}$ maps each symbol ω^{k} to the value of the k-th argument when the current function was called.

Since a symbol denotes an argument *value*, its meaning does not depend on the heap and never changes while the callee runs.

Definition 1.2 (Symbolic Type Denotation). The set of values $[\![\tau^\omega]\!]^\eta_h \subseteq \mathbb{V}$ described by a symbolic type τ^ω is defined as follows:

$$\begin{aligned} v \in [\![\tau]\!]^\eta_h &\iff (h, v) \in \gamma_\tau(\tau) \\ v \in [\![\omega]\!]^\eta_h &\iff v = \eta(\omega) \\ v \in [\![\tau^\omega.\mathtt{f}]\!]^\eta_h &\iff \exists a.\ a \in [\![\tau^\omega]\!]^\eta_h \wedge h(a)(\mathtt{f}) = v \\ a \in [\![\{\overline{\mathtt{f}_k : \tau^\omega_k}\}]\!]^\eta_h &\iff \forall k.\ \mathtt{f}_k \in \mathrm{dom}(h(a)) \wedge h(a)(\mathtt{f}_k) \in [\![\tau^\omega_k]\!]^\eta_h \end{aligned}$$

Definition 1.3 (Satisfaction of Type Propositions and Type Guards).

$$\begin{aligned} (h, \rho, \eta) \models \top_\psi &\quad \text{always} \qquad (h, \rho, \eta) \models \bot_\psi \text{ never} \\ (h, \rho, \eta) \models x : \tau &\iff x \in \mathrm{dom}(\rho) \wedge (h, \rho(x)) \in \gamma_\tau(\tau) \\ (h, \rho, \eta) \models \omega <: \tau &\iff (h, \eta(\omega)) \in \gamma_\tau(\tau) \\ (h, \rho, \eta) \models \psi_1 \wedge \psi_2 &\iff (h, \rho, \eta) \models \psi_1 \wedge (h, \rho, \eta) \models \psi_2 \\ (h, \rho, \eta, v) \models G &\iff \forall \tau^T \in \mathcal{T}.\ (h, v) \in \gamma_\tau(\tau^T) \Rightarrow (h, \rho, \eta) \models G(\tau^T) \end{aligned}$$

A type guard is a set of conditional promises: if the value belongs to a target type τ^T, the proposition $G(\tau^T)$ holds. Both the membership and the proposition are judged in the *same* heap h. This flow-sensitive reading is why the analysis weakens both type propositions and type guards when the heap changes.

Definition 1.4 (Abstract Values and Environments).

$$\begin{aligned} v \in \gamma^\eta_{h,\rho}(\tau^\omega[G]) &\iff v \in [\![\tau^\omega]\!]^\eta_h \wedge (h, \rho, \eta, v) \models G \\ \rho \in \gamma^\eta_h(\widehat{\rho}) &\iff \forall x \in \mathrm{dom}(\rho).\ x \in \mathrm{dom}(\widehat{\rho}) \wedge \rho(x) \in \gamma^\eta_{h,\rho}(\widehat{\rho}(x)) \end{aligned}$$

Definition 1.5 (Heap Evolution). A heap h evolves into h' within an effect δ, written $h \preceq_\delta h'$, if every address allocated in h is still allocated in h', and every field $\mathtt{f}$ of such an address whose value differs between h and h' satisfies $\mathtt{f} \in \mathrm{dom}(\delta)$ and $(h', h'(a)(\mathtt{f})) \in \gamma_\tau(\delta(\mathtt{f}))$.

Definition 1.6 (Frame Coverage). A typing result $\Gamma = (\widehat{\rho}, \psi, \delta)$ covers a memory $m = (h, \rho)$ with a symbol valuation η and an entry heap h_0, written $(\eta, h_0) \vdash m : \Gamma$, if $\rho \in \gamma^\eta_h(\widehat{\rho})$, $(h, \rho, \eta) \models \psi$, and $h_0 \preceq_\delta h$.

The last condition makes the effect δ of a typing result an over-approximation of all field writes since the entry of the current function. It is the invariant that justifies applying the callee effect to the caller when returning from a function.

Definition 1.7 (State Coverage). An analysis result $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ covers a concrete state $\sigma = (\kappa, l, m)$ with an initial heap h_{init}, written $\sigma \in \gamma(\widehat{\sigma})$, if

(1) m is well-formed (every address stored in the environment or the heap is allocated, and every function value belongs to the program);
(2) there exists a well-formed symbol valuation η such that $(\eta, h_0) \vdash m : \xi(l)$ where h_0 is the entry heap of the frame, and if $\kappa = (\kappa_c, l_c, m_c)$, then $\eta(\omega^{\mathsf{k}})$ is the value of the k-th argument of the call at l_c evaluated in m_c;
(3) if $\kappa = (\kappa_c, l_c, m_c)$, then $l_c \in \widehat{\kappa}(\Lambda)$ for the function Λ containing l, and the caller state (κ_c, l_c, m_c) is covered by $\widehat{\sigma}$ as well.

The recursion in the last condition covers every frame on the call stack, each with its own symbol valuation and entry heap.

1.4.3 Valid Analysis Results. Instead of the least fixpoint of the abstract semantics, we state soundness for any analysis result that satisfies the transfer of every instruction as a constraint. This formulation only requires the post-fixpoint property that the proof actually uses.

Definition 1.8 (Semantic Order). $\Gamma_1 \sqsubseteq \Gamma_2$ if $(\eta, h_0) \vdash m : \Gamma_1$ implies $(\eta, h_0) \vdash m : \Gamma_2$ for all η, h_0, and m.

Definition 1.9 (Valid Analysis Result). An analysis result $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ is valid for a program P, written $\mathsf{Valid}(P, \widehat{\sigma})$, if the following holds for every label l with $\Gamma = \xi(l)$ and its next label l':

$$
\begin{array}{ll}
x = e & : \Gamma.\widehat{\mathbf{update}}_x(x, \widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma)) \sqsubseteq \xi(l') \\
x = \{\overline{\mathtt{f}_k : e_k}\} & : \Gamma.\widehat{\mathbf{update}}_x(x, \{\mathtt{f}_k : \widehat{[\![e_k]\!]}^{\mathrm{g}}_e(\Gamma)\}.\widehat{\mathbf{bind}}(\psi)) \sqsubseteq \xi(l') \\
e_1.\mathtt{f} = e_2 & : \Gamma.\widehat{\mathbf{update}}_{\mathtt{f}}(_, \mathtt{f}, \widehat{[\![e_2]\!]}^{\mathrm{g}}_e(\Gamma)) \sqsubseteq \xi(l') \\
\mathtt{if}\ (e)\ l_t\ \mathtt{else}\ l_f & : \Gamma.\widehat{\mathbf{refine}}(e, \#\mathtt{t}) \sqsubseteq \xi(l_t) \wedge \Gamma.\widehat{\mathbf{refine}}(e, \top \backslash \#\mathtt{t}) \sqsubseteq \xi(l_f) \\
\mathtt{assert}\ e & : \Gamma.\widehat{\mathbf{refine}}(e, \#\mathtt{t}) \sqsubseteq \xi(l') \\
x = e_0(\overline{e_k}) & : \forall \Lambda = \lambda(\overline{x_k}).l_\Lambda \in \mathsf{MayCall}(\Gamma, e_0).\ l \in \widehat{\kappa}(\Lambda) \wedge \\
& :\quad (\{\overline{x_k \mapsto \omega^{\mathsf{k}}}\}, \psi', \bot_\delta) \sqsubseteq \xi(l_\Lambda) \\
\mathtt{return}\ e & : \forall l_c \in \widehat{\kappa}(\mathsf{func}(l)). \\
& :\quad \Gamma_c.\widehat{\mathbf{update}}_x(x_c, \widehat{[\![e]\!]}^{\mathrm{g}}_e(\Gamma).\mathbf{instantiate}(\xi(l_c), \overline{e_{c,j}}, \delta).\widehat{\mathbf{bind}}(\Gamma_c.\psi)) \sqsubseteq \xi(\mathsf{next}(l_c))
\end{array}
$$

where $\psi' = \bigwedge_k \omega^{\mathsf{k}} <: \lceil \tau^\omega_k \rceil_\psi$ for $\widehat{[\![e_k]\!]}^{\mathrm{g}}_e(\Gamma) = \tau^\omega_k[_]$, $\mathsf{MayCall}(\Gamma, e_0)$ is the set of functions that e_0 may evaluate to in a memory covered by Γ, δ is the effect of Γ, $x_c = _(\overline{e_{c,j}})$ is the instruction at l_c, and $\Gamma_c = \xi(l_c).\widehat{\mathbf{apply}}(\delta)$.

The return constraint reflects the revised instantiation in Figure 9 and §1.3.2: the argument descriptions and guards come from the caller state $\xi(l_c)$ at the call, the callee effect δ weakens the propagated propositions, and the result is bound with the caller proposition after applying δ.

Valid analysis results always exist: the analysis result that maps every label to $(\lambda x.\top, \top_\psi, \lambda \mathtt{f}.\top)$ and every function to all labels is valid for every program, and it covers every well-formed initial state of the main function (`topAnalysis_valid`, `topAnalysis_covers_main`).

1.4.4 Soundness Theorem.

Theorem 1.10 (Soundness). *Let P be a well-formed program and $\widehat{\sigma}$ an analysis result with $\mathit{Valid}(P, \widehat{\sigma})$. If $\sigma_0 \in \gamma(\widehat{\sigma})$ with the initial heap of σ_0 and $\sigma_0 \rightsquigarrow^* \sigma$, then $\sigma \in \gamma(\widehat{\sigma})$.*

Corollary 1.11 (Type Soundness of Variables). *Under the assumptions of Theorem 1.10, if $\sigma = (_, l, (h, \rho))$ is reachable, $\rho(x) = v$, and $\xi(l) = (\widehat{\rho}, \psi, _)$ with $\widehat{\rho}(x) = \tau^\omega[_]$, then $(h, v) \in \gamma_\tau(\lceil \tau^\omega \rceil_\psi)$.*

The theorem is a *coverage* soundness: the analysis result over-approximates every reachable state. Well-formedness of a program only requires that labels and functions are consistent (e.g., every branch target belongs to the same function); it does not require the absence of type errors.

1.4.5 Proof Structure. Theorem 1.10 follows by induction on $\rightsquigarrow^*$ from the preservation of $\gamma(\widehat{\sigma})$ by a single transition (`step_preserves`), and Corollary 1.11 follows from the soundness of upper type bounds (`upperTy_sound`). The preservation is proved by case analysis on the instruction with the following groups of lemmas.

(1) **Transport.** If $h \preceq_\delta h'$, the weakening $\mathbf{weaken}_\delta$ preserves the meaning of types, type propositions, type guards, symbolic types, and abstract environments from h to h'. Heap evolution is reflexive, holds for a single field write $[\mathtt{f} \mapsto \tau]$ when the written value has type τ, and composes with the effect composition $\delta \fatsemi \delta'$. This group justifies the field update and the effect application.

(2) **Type Guard Operations.** Guard lookup, update, refinement, and join are sound with respect to guard satisfaction. The query-form field lookup $G.\mathtt{f}$ is sound because $h(a)(\mathtt{f}) \in \gamma_\tau(\tau^T)$ implies $a \in \gamma_\tau(\{\mathtt{f} : \tau^T\})$ for open records, and the record type guard is sound because a record in τ^T has field values in $\tau^T.\mathtt{f}_j$ (`tyField_sound`).
(3) **Expressions.** If a memory is covered by Γ and e evaluates to v, then $v \in \gamma^{\eta}_{h,\rho}(\widehat{[\![e]\!]}_e(\Gamma))$. This relies on the soundness of the type proposition inference $\widehat{\mathbf{infer}}_\psi$ (both the symbolic and the expression derivations) and of $\widehat{\mathbf{bind}}$.
(4) **Updates and Refinement.** $\widehat{\mathbf{update}}_x$, $\widehat{\mathbf{update}}_\mathtt{f}$, and $\widehat{\mathbf{refine}}$ are sound: after a variable update, propositions about the variable are removed; after a field update, the typing result is weakened with the single-field effect; and refinement only adds propositions that hold in the taken branch.
(5) **Calls and Returns.** A call establishes the coverage of the callee entry with η mapping each symbol to the argument value and with the caller heap as the entry heap. For a return, the callee coverage gives $h_c \preceq_\delta h_r$ from the caller heap h_c at the call to the heap h_r at the return. The stability lemma shows that the symbolic type of each weakened argument value $\widehat{v}_k$ still denotes the argument value in h_r, and the instantiation lemma combines it with transport: the argument guards weakened by δ hold in h_r, and each callee proposition $\omega^{\mathsf{k}} <: \tau$, which holds for the argument value, is translated by $\widehat{\mathbf{infer}}_\psi$ into a caller proposition that holds in h_c and, after $\mathbf{weaken}_\delta$, in h_r. Finally, applying δ to the caller typing result and binding its proposition yields the coverage of the return site.

1.4.6 Relation to the Implementation. The theorem covers the formalized analysis, and we do not formally verify that the implementation follows it. The implementation differs from the formalization in three ways.

Precision Refinements. The implementation uses more information than the formalization at several points, each justified by the same argument as the corresponding lemma: 1) it looks up field guards with $\{\mathtt{f} : \tau^T\} \sqcap \lceil \tau^\omega \rceil_\psi$, where τ^ω is the symbolic type of the base record; 2) it keeps guard entries for record target types whose fields are not in the effect; 3) it records effects per pair of a record type name and a field name using the partitioned abstract addresses in §4 of the paper.

Typed Callees. For a callee whose declared return type does not overlap with target types, the caller uses the declared return type as the return value instead of the analyzed one, while it still applies the callee effect, and the analysis checks this declared type at every return of the callee (§4.1 of the paper).

Soundy Approximation. The implementation makes one approximation inherited from the original implementation: a mutation of a list or a map through an alias weakens only the propositions on the mutated variable, and the analysis does not narrow the element types of lists and maps. This approximation may miss type errors, and replacing it with the formalized operations is future work.

2 Provenance Tracking

Section 4.3 of the paper summarizes provenance tracking, and this section gives its definitions.

In this section, we introduce the provenance, which augments type propositions to add an explanation of why the type proposition can ensure that refinement is legal.

$$\tau_{\zeta_1} \sqsubseteq \tau_{\zeta_2} \Leftrightarrow \text{if } \tau_1 \neq \tau_2 \text{ then } \tau_1 \sqsubseteq \tau_2 \text{ else } \begin{cases} \zeta_1.\text{child} \sqsubseteq \zeta_2.\text{child} & \text{if } \zeta_1 = \zeta_2 = \text{Root}(l, _) \mid \text{Call}(l, _) \mid \text{Join}(l, _) \mid \text{Meet}(l, _) \\ \texttt{\#t} & \text{if } \zeta_1 = \zeta_2 = \text{Leaf}(l) \\ \texttt{\#f} & \text{otherwise} \end{cases}$$

(a) Partial order $\sqsubseteq : \mathcal{T}_{\mathbb{Z}} \times \mathcal{T}_{\mathbb{Z}} \to \mathbb{B}$

$$\tau_{\zeta_1} \sqcup \tau_{\zeta_2} = \text{canonical}\left(\begin{cases} \tau_{\zeta_1} & \text{if } \tau_{\zeta_2} \sqsubseteq \tau_{\zeta_1} \\ \tau_{\zeta_2} & \text{if } \tau_{\zeta_1} \sqsubseteq \tau_{\zeta_2} \\ (\tau_1 \sqcup \tau_2)_{(\zeta_1 \sqcup \zeta_2)} & \text{otherwise} \end{cases}\right)$$

$$\text{where } \zeta_1|_{\tau_1} \sqcup \zeta_2|_{\tau_2} = \begin{cases} \zeta_1 & \text{if } \zeta_1 = \zeta_2 \\ \text{Join}(s_1 \sqcup s_2) & \text{if } \zeta_1 = \text{Join}(s_1) \wedge \zeta_2 = \text{Join}(s_2) \\ \text{Join}(s_1 \sqcup \{\tau_{\zeta_2}\}) & \text{if } \zeta_1 = \text{Join}(s_1) \wedge \zeta_2 = \text{Call} \mid \text{Leaf} \\ \text{Join}(s_2 \sqcup \{\tau_{\zeta_1}\}) & \text{if } \zeta_2 = \text{Join}(s_2) \wedge \zeta_1 = \text{Call} \mid \text{Leaf} \\ \text{Join}(\{\tau_{\zeta_1}, \tau_{\zeta_2}\}) & \text{otherwise} \end{cases}$$

(b) Join operation $\sqcup : \mathcal{T}_{\mathbb{Z}} \times \mathcal{T}_{\mathbb{Z}} \to \mathcal{T}_{\mathbb{Z}}$

$$\text{canonical}(\text{Join}(\{\overline{\tau_\zeta}\})) = \begin{cases} \text{canonical}(\text{Join}(\{\overline{\tau_\zeta}\} \setminus \{\tau_{\zeta_1}\})) \\ \quad \text{if } \exists \tau_{\zeta_2} \in \{\overline{\tau_\zeta}\} \setminus \{\tau_{\zeta_1}\} \text{ s.t. } (\tau_1 <: \tau_2 \wedge \tau_1 \neq \tau_2) \\ \text{Join}(\{\overline{\text{canonical}(\tau_\zeta)}\}) \quad \text{otherwise} \end{cases}$$

$$\text{canonical}(\tau_\zeta) = \tau_\zeta \quad \text{for } \zeta \neq \text{Join}(_) \wedge \zeta \neq \text{Meet}(_)$$

(c) Auxiliary operation canonical : $\mathcal{T}_{\mathbb{Z}} \to \mathcal{T}_{\mathbb{Z}}$

Fig. 12. Domain operations for types with provenance.

2.1 Definition

A *provenance* is bound with an *explanation* that shows why the refinement can happen; at the same time, it is an abstraction of the propagation path of propositions as a citizen of the abstract domain. It gives the simplest possible call paths set for each variable and each refinement, that affected the type guard used at the point.

Provenance	$\zeta \in \mathbb{Z}$	=	Nodes
Type With Provenance	$\tau_\zeta \in \mathcal{T}_{\mathbb{Z}}$	=	$\mathcal{T} \times \mathbb{Z}$
Type Propositions	$\psi \in \Psi$	=	$(\mathcal{X} \uplus \Omega) \to \mathcal{T}_{\mathbb{Z}}$

$$\text{Nodes} \ni \zeta = \text{Root}(l, \tau_\zeta) \mid \text{Call}(l, \tau_\zeta) \mid \text{Join}(l, \{\overline{\tau_\zeta}\}) \mid \text{Meet}(l, \{\overline{\tau_\zeta}\}) \mid \text{Leaf}(l) \mid \bot \mid \top$$

The type with provenance is same as the type in the section before, but augmented with the provenance tree. The provenance tree are made of multiple kinds of nodes represents the path in term of the analysis point, which means a set of locations we want to track the action (change of the type guard). Root, Call has a single refinement point with a single child, and Join, Meet has a set of children which can be retrieved by ζ.child. Most of definitions are trivial, but we note that Root means the refinement point and Leaf means the generation point of the guard.

2.2 Domain Operations

Provenances are not simple logs but a part of abstract domain. This means, if a pair of provenances are merged, we should demonstrate that a provenance represents the simplest path that can explain both provenances. We define here an operation between types with provenance as Figure 12, with a slight abuse of annotation with $\tau_{\zeta_i} = (\tau_i, \zeta_i)$. Since rules for $\bot$ and $\top$ are trivial, we omit them.

Abstract States	$\widehat{\sigma} \in \widehat{\Sigma}$	$=$	$\widehat{\mathbb{K}} \times \Xi$
Abstract Calling Contexts	$\widehat{\kappa} \in \widehat{\mathbb{K}}$	$=$	$\mathcal{F} \to \mathcal{P}(\mathcal{L})$
Flow-Sensitive Typing Results	$\xi \in \Xi$	$=$	$\mathcal{L} \to \mathbb{\Gamma}$
Typing Results	$\Gamma \in \mathbb{\Gamma}$	$=$	$\widehat{\mathbb{E}} \times \Delta$
Abstract Environments	$\widehat{\rho} \in \widehat{\mathbb{E}}$	$=$	$\mathcal{X} \to \widehat{\mathbb{V}}$
Abstract Values	$\widehat{v} \in \widehat{\mathbb{V}}$	$=$	$\mathcal{T}$
Effects	$\delta \in \Delta$	$=$	$\mathbb{F} \to \mathcal{T}$

Fig. 13. Abstract domains for each component of the type analysis.

Note that it omits unnecessary nodes if a simpler explanation is possible, let the $\zeta_1 = (\mathbf{N}, \mathsf{Call}(l_1))$ and $\zeta_2 = (\mathbf{N} \mid \mathbf{S}, \mathsf{Call}(l_2))$, then $\zeta_1 \sqcup \zeta_2$ ignores the former since the latter is enough to explain the refinement. We do not explain each operation in detail due to the limit of space, but the essence of the provenance is stated in §5.3.2 of the paper.

3 Base Type Analysis

This section gives the type analysis of JSTAR that §1 extends: its abstract domains, its abstract semantics without type guards, and its syntactic type refinement. The green parts of the figures in §1 replace the corresponding parts here.

We first define flow-sensitive abstract states and then define abstract semantics for the type analysis on a given $\mathsf{IR}_{\mathsf{ES}}$ program.

Abstract Domains. Figure 13 shows the abstract domains for each component of the type analysis. An abstract state $\widehat{\sigma} \in \widehat{\Sigma}$ is a pair of an abstract calling context and a flow-sensitive typing result. An abstract calling context $\widehat{\kappa} \in \widehat{\mathbb{K}}$ is a mapping from each function to a set of call-site labels. A flow-sensitive typing result $\xi \in \Xi$ is a mapping from labels to typing results. A typing result $\Gamma \in \mathbb{\Gamma}$ is a pair of an abstract environment and an effect. An abstract environment $\widehat{\rho} \in \widehat{\mathbb{E}}$ is a mapping from variables to abstract values. An abstract value $\widehat{v} \in \widehat{\mathbb{V}}$ is a type. Finally, an effect $\delta \in \Delta$ is a mapping from fields to types that represents which fields may be updated with new types in the current calling context. Effects are used to propagate the may-update information of fields into the calling context when returning from a function call for soundness. The partial order $\sqsubseteq$, join $\sqcup$, meet $\sqcap$ operations, and the remove $\setminus$ operation are defined in a point-wise manner for each component of the abstract domain. We define operations (concretization (γ), partial order ($\sqsubseteq$), bottom ($\bot$), join ($\sqcup$), and meet ($\sqcap$)) on abstract domains in a point-wise manner. Their definitions are defined in Figure 14.

Abstract Semantics. The abstract semantics $\widehat{[\![P]\!]}$ of a program is defined as a fixed-point computation of the abstract transfer function $\widehat{F} : \widehat{\Sigma} \to \widehat{\Sigma}$ from an initial abstract state $\widehat{\sigma}_\iota$:

$$\widehat{[\![P]\!]} = \mathbf{lfp}\widehat{F} = \lim_{n \to \infty} (\widehat{F})^n(\bot_\sigma) \qquad \text{where} \qquad \widehat{F}(\widehat{\sigma}) = \widehat{\sigma}_\iota \sqcup \left(\bigsqcup_{l \in \mathcal{L}} \widehat{[\![\mathsf{inst}(l)]\!]}_i(l, \widehat{\sigma}) \right)$$

where $\widehat{\sigma}_\iota$ satisfies the constraints $\Sigma_\iota \subseteq \gamma_\sigma(\widehat{\sigma}_\iota)$ and $\widehat{[\![-]\!]}_i$ and $\widehat{[\![-]\!]}_e$ are the abstract semantics of instructions and expressions, respectively, defined in Figure 15.

For abstract semantics, we introduce the auxiliary operations for typing results $\Gamma = (\widehat{\rho}, \delta) \in \mathbb{\Gamma}$:

$$\gamma_\sigma : \widehat{\Sigma} \to \mathcal{P}(\Sigma) \quad \gamma_\kappa : \widehat{\mathbb{K}} \to \mathcal{P}(\Sigma) \quad \gamma_\xi : \Xi \to \mathcal{P}(\Sigma) \quad \gamma_\Gamma : \mathbb{\Gamma} \to \mathcal{P}(\Sigma)$$
$$\gamma_\rho : \widehat{\mathbb{E}} \to \mathcal{P}(\mathbb{M}) \quad \gamma_v : \widehat{\mathbb{V}} \to \mathcal{P}(\mathbb{H} \times \mathbb{V}) \quad \gamma_\delta : \Delta \to \mathcal{P}(\Sigma)$$

$$\begin{aligned}
\gamma_\sigma(\widehat{\sigma}) &= \{\sigma \mid \widehat{\sigma} = (\widehat{\kappa}, \xi) \wedge \sigma \in \gamma_\kappa(\widehat{\kappa}) \wedge \sigma \in \gamma_\xi(\xi)\} \\
\gamma_\kappa(\widehat{\kappa}) &= \{\sigma \mid \kappa = \sigma.\text{caller} \wedge (\kappa = \epsilon \vee (\kappa.\text{state} \in \gamma_\kappa(\widehat{\kappa}) \wedge (\kappa.\text{label}, \kappa.\text{var}) \in \widehat{\kappa} \circ \text{func}(l)))\} \\
\gamma_\xi(\xi) &= \{\sigma \mid \sigma \in \gamma_\Gamma \circ \xi(\sigma.\text{label}) \wedge \kappa = \sigma.\text{caller} \wedge (\kappa = \epsilon \vee \kappa.\text{state} \in \gamma_\xi(\xi))\} \\
\gamma_\Gamma(\Gamma) &= \{\sigma \mid \Gamma = (\widehat{\rho}, \delta) \wedge \sigma.\text{mem} \in \gamma_\rho(\widehat{\rho}) \wedge \sigma \in \gamma_\delta(\delta)\} \\
\gamma_\rho(\widehat{\rho}) &= \{m \mid m = (h, \rho) \wedge \forall (x \mapsto v) \in \rho.\ (h, v) \in \gamma_v \circ \widehat{\rho}(x)\} \\
\gamma_v(\tau) &= \gamma_\tau(\tau) \\
\gamma_\delta(\delta) &= \{\sigma \mid \kappa = \sigma.\text{caller} \wedge (\kappa = \epsilon \vee (\kappa.\text{heap}, \sigma.\text{heap}) \in_\delta \delta)\}
\end{aligned}$$

where $(h, h') \in_\delta \delta \iff \forall a \mapsto r \in h.\ \forall \mathsf{f} \mapsto v \in r.\ v' = h'(a)(\mathsf{f}) \wedge (v' \neq v \Rightarrow (h', v') \in \gamma_\tau \circ \delta(\mathsf{f}))$

$$\widehat{d_1} \sqsubseteq \widehat{d_2} \iff \gamma_d(\widehat{d_1}) \subseteq \gamma_d(\widehat{d_2})$$

$$\begin{aligned}
&\bot_\sigma = (\bot_\kappa, \bot_\xi) && \bot_\xi = \{l \mapsto \bot_\Gamma \mid l \in \mathcal{L}\} && \bot_\rho = \{x \mapsto \bot_v \mid x \in \mathcal{X}\} && \bot_v = \bot \\
&\bot_\kappa = \{\Lambda \mapsto \varnothing \mid \Lambda \in \mathcal{F}\} && \bot_\Gamma = (\bot_\rho, \bot_\delta) && \bot_\delta = \{\mathsf{f} \mapsto \bot \mid \mathsf{f} \in \mathbb{F}\}
\end{aligned}$$

$$\Gamma_1 \sqcup \Gamma_2 = (\widehat{\rho}_1 \sqcup \widehat{\rho}_2, \delta_1 \sqcup \delta_2) \text{ where } \Gamma_1 = (\widehat{\rho}_1, \delta_1) \text{ and } \Gamma_2 = (\widehat{\rho}_2, \delta_2)$$

$$\begin{aligned}
\widehat{\sigma}_1 \sqcup \widehat{\sigma}_2 &= (\widehat{\kappa}_1 \sqcup \widehat{\kappa}_2, \xi_1 \sqcup \xi_2) \\
\widehat{\kappa}_1 \sqcup \widehat{\kappa}_2 &= \{\Lambda \mapsto \widehat{\kappa}_1(\Lambda) \cup \widehat{\kappa}_2(\Lambda) \mid \Lambda \in \mathcal{F}\} \\
\xi_1 \sqcup \xi_2 &= \{l \mapsto \xi_1(l) \sqcup \xi_2(l) \mid l \in \mathcal{L}\} \\
\widehat{\rho}_1 \sqcup \widehat{\rho}_2 &= \{x \mapsto \widehat{\rho}_1(x) \sqcup \widehat{\rho}_2(x) \mid x \in \mathcal{X}\} \\
\delta_1 \sqcup \delta_2 &= \{\mathsf{f} \mapsto \delta_1(\mathsf{f}) \sqcup \delta_2(\mathsf{f}) \mid \mathsf{f} \in \mathbb{F}\} \\
\widehat{v}_1 \sqcup \widehat{v}_2 &= \widehat{v}_1 \sqcup \widehat{v}_2
\end{aligned}$$

$$\Gamma_1 \sqcap \Gamma_2 = (\widehat{\rho}_1 \sqcap \widehat{\rho}_2, \delta_1 \sqcap \delta_2) \text{ where } \Gamma_1 = (\widehat{\rho}_1, \delta_1) \text{ and } \Gamma_2 = (\widehat{\rho}_2, \delta_2)$$

$$\begin{aligned}
\widehat{\sigma}_1 \sqcap \widehat{\sigma}_2 &= (\widehat{\kappa}_1 \sqcap \widehat{\kappa}_2, \xi_1 \sqcap \xi_2) \\
\widehat{\kappa}_1 \sqcap \widehat{\kappa}_2 &= \{\Lambda \mapsto \widehat{\kappa}_1(\Lambda) \cup \widehat{\kappa}_2(\Lambda) \mid \Lambda \in \mathcal{F}\} \\
\xi_1 \sqcap \xi_2 &= \{l \mapsto \xi_1(l) \sqcap \xi_2(l) \mid l \in \mathcal{L}\} \\
\widehat{\rho}_1 \sqcap \widehat{\rho}_2 &= \{x \mapsto \widehat{\rho}_1(x) \sqcap \widehat{\rho}_2(x) \mid x \in \mathcal{X}\} \\
\delta_1 \sqcap \delta_2 &= \{\mathsf{f} \mapsto \delta_1(\mathsf{f}) \sqcap \delta_2(\mathsf{f}) \mid \mathsf{f} \in \mathbb{F}\} \\
\widehat{v}_1 \sqcap \widehat{v}_2 &= \widehat{v}_1 \sqcap \widehat{v}_2
\end{aligned}$$

Fig. 14. Concretization (γ), partial order ($\sqsubseteq$), bottom ($\bot$), join ($\sqcup$), and meet ($\sqcap$) operations on abstract domains.

- $\Gamma.\widehat{\textbf{update}}_x : (\mathcal{X} \times \widehat{\mathbb{V}}) \to \mathbb{\Gamma}$ updates a variable in a typing result with a new abstract value.

$$\Gamma.\widehat{\textbf{update}}_x(x, \widehat{v}) = (\widehat{\rho}[x \mapsto \widehat{v}], \delta)$$

- $\Gamma.\widehat{\textbf{update}}_\mathsf{f} : (\widehat{\mathbb{V}} \times \mathbb{F} \times \widehat{\mathbb{V}}) \to \mathbb{\Gamma}$ updates a field of an abstract value in a typing result.

$$\Gamma.\widehat{\textbf{update}}_\mathsf{f}(_, \mathsf{f}, \widehat{v}_2) = (\{x \mapsto (\widehat{v} \sqcup \widehat{v}[\mathsf{f} : \widehat{v}_2]) \mid (x \mapsto \widehat{v}) \in \widehat{\rho}\}, \delta[\mathsf{f} : \widehat{v}_2])$$

- $\Gamma.\widehat{\textbf{apply}} : \Delta \to \mathbb{\Gamma}$ applies an effect (may-update information of fields) to a typing result.

$$\Gamma.\widehat{\textbf{apply}}(\delta) = \bigsqcup\{\Gamma.\widehat{\textbf{update}}_\mathsf{f}(\top_{\widehat{v}}, \mathsf{f}, \tau) \mid (\mathsf{f} : \tau) \in \delta\}$$

$$\widehat{[\![-]\!]}_i : (\mathcal{L} \times \widehat{\Sigma}) \to \widehat{\Sigma}$$

$$\begin{array}{lll}
\widehat{[\![x = e]\!]}_i(l, \widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{update}}_x(x, \widehat{[\![e]\!]}_e(\Gamma)) \\
\widehat{[\![x = \{\mathtt{f}_k : e_k\}]\!]}_i(l, \widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{update}}_x(x, \{\mathtt{f}_k : \widehat{[\![e_k]\!]}_e(\Gamma)\}) \\
\widehat{[\![e_1.\mathtt{f} = e_2]\!]}_i(l, \widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{update}}_{\mathtt{f}}(\widehat{[\![e_1]\!]}_e(\Gamma), \mathtt{f}, \widehat{[\![e_2]\!]}_e(\Gamma)) \\
\widehat{[\![\mathtt{if}\ (e)\ l_1\ \mathtt{else}\ l_2]\!]}_i(l, \widehat{\sigma}) & = (\varnothing, \{l_t \mapsto \Gamma_t, l_f \mapsto \Gamma_f\}) & \text{where } \begin{cases} \Gamma_t = \Gamma.\widehat{\mathbf{refine}}(e, \#\mathtt{t}) \\ \Gamma_f = \Gamma.\widehat{\mathbf{refine}}(e, \#\mathtt{f}) \end{cases} \\
\widehat{[\![\mathtt{assert}(e)]\!]}_i(l, \widehat{\sigma}) & = (\varnothing, \{\mathsf{next}(l) \mapsto \Gamma_1\}) & \text{where } \Gamma_1 = \Gamma.\widehat{\mathbf{refine}}(e, \#\mathtt{t}) \\
\widehat{[\![\mathtt{call}\ x = e_0(\overline{e_k})]\!]}_i(l, \widehat{\sigma}) & = (\widehat{\kappa}_1, \{\overline{l_j \mapsto \Gamma_j}\}) & \text{where } \begin{cases} \widehat{v}_0 = \widehat{[\![e_0]\!]}_e(\Gamma) \\ \{\overline{\Lambda_j}\} = \{\overline{\lambda(\overline{x_{j,k}}).l_j}\} = \gamma_v(\widehat{v}_0) \cap \mathcal{F} \\ \Gamma_j = (\{x_{j,k} \mapsto \widehat{[\![e_k]\!]}_e(\Gamma)\}, \bot_\delta) \\ \widehat{\kappa}_1 = \{\Lambda_j \mapsto \{l\}\} \end{cases} \\
\widehat{[\![\mathtt{return}\ e]\!]}_i(l, \widehat{\sigma}) & = (\varnothing, \{\overline{\mathsf{next}(l_k) \mapsto \Gamma'_k}\}) & \text{where } \begin{cases} \{\overline{l_k}\} = \widehat{\kappa} \circ \mathsf{func}(l) \\ \mathsf{inst}(l_k) = \mathtt{call}\ x_k = _; \\ \Gamma_k = \xi(l_k).\widehat{\mathbf{apply}}(\delta) \\ \Gamma'_k = \Gamma_k.\widehat{\mathbf{update}}_x(x_k, \widehat{[\![e]\!]}_e(\Gamma)) \end{cases}
\end{array}$$

$$\widehat{[\![-]\!]}_e : \mathbb{\Gamma} \to \widehat{\mathbb{V}}$$

$$\widehat{[\![p]\!]}_e(\Gamma) = p \quad \widehat{[\![x]\!]}_e(\Gamma) = \widehat{\rho}(x) \quad \widehat{[\![\ominus e]\!]}_e(\Gamma) = \ominus\widehat{[\![e]\!]}_e(\Gamma) \quad \widehat{[\![e : \tau]\!]}_e(\Gamma) = \mathbf{B}$$
$$\widehat{[\![\Lambda]\!]}_e(\Gamma) = \Lambda \quad \widehat{[\![e.\mathtt{f}]\!]}_e(\Gamma) = \widehat{[\![e]\!]}_e(\Gamma).\mathtt{f} \quad \widehat{[\![e_1 \oplus e_2]\!]}_e(\Gamma) = \widehat{[\![e_1]\!]}_e(\Gamma) \oplus \widehat{[\![e_2]\!]}_e(\Gamma)$$

Fig. 15. Abstract semantics of instructions and expressions of $\mathsf{IR}_{\mathsf{ES}}$ where $\widehat{\sigma} = (\widehat{\kappa}, \xi)$ and $\xi(l) = \Gamma = (\widehat{\rho}, \delta)$.

- $\Gamma.\widehat{\mathbf{refine}} : (\mathcal{E} \times \mathbb{B}) \to \mathbb{\Gamma}$ performs a *syntactic type refinement* on a typing result in branch/assertion instructions with conditional expressions as highlighted in Figure 15.

$$\Gamma.\widehat{\mathbf{refine}}(e_1\ \&\&\ e_2, b) = \begin{cases} \Gamma_1 \sqcap \Gamma_2 \text{ if } b \\ \Gamma_1 \sqcup \Gamma_2 \text{ if } \neg b \end{cases} \qquad \Gamma.\widehat{\mathbf{refine}}(!\,e, b) = \Gamma.\widehat{\mathbf{refine}}(e, \neg b)$$
$$\Gamma.\widehat{\mathbf{refine}}(e_1\ ||\ e_2, b) = \begin{cases} \Gamma_1 \sqcup \Gamma_2 \text{ if } b \\ \Gamma_1 \sqcap \Gamma_2 \text{ if } \neg b \end{cases} \qquad \Gamma.\widehat{\mathbf{refine}}(x : \tau, b) = \begin{cases} (\widehat{\rho}[x \mapsto \widehat{\rho}(x) \sqcap \tau], \delta) \text{ if } b \\ (\widehat{\rho}[x \mapsto \widehat{\rho}(x) \setminus \tau], \delta) \quad \text{if } \neg b \end{cases}$$

  where $\Gamma_1 = \Gamma.\widehat{\mathbf{refine}}(e_1, b)$ and $\Gamma_2 = \Gamma.\widehat{\mathbf{refine}}(e_2, b)$.

References

[1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming languages (POPL)*. doi:10.1145/512950.512973

[2] Patrick Cousot and Radhia Cousot. 1992. Abstract Interpretation Frameworks. *Journal of Logic and Computation (JLC)* 2, 4 (1992), 511–547. doi:10.1093/logcom/2.4.511

[3] Jihyeok Park, Seungmin An, Wonho Shin, Yusung Sim, and Sukyoung Ryu. 2021. JSTAR: JavaScript Specification Type Analyzer using Refinement. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1109/ASE51524.2021.9678781

[4] Jihyeok Park, Jihee Park, Seungmin An, and Sukyoung Ryu. 2020. JISET: JavaScript IR-based Semantics Extraction Toolchain. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 647–658. doi:10.1145/3324884.3416632